

Product Requirements Document: Next-Generation OS Development Language

Author: Manus AI

Version: 1.0

Date: June 12, 2026

1. Introduction

This Product Requirements Document (PRD) outlines the vision, scope, and technical specifications for a new programming language designed from the ground up for operating system development. The primary goal is to create a unified, highly efficient, fast, and secure systems programming language that eliminates the historical necessity of mixing Assembly and C, providing unparalleled low-level hardware control through a clean, high-level, and intuitive syntax. This language aims to be powerful enough to enable the creation of full-scale, production-grade operating systems, including those with the complexity of Windows, and to ultimately supersede existing systems programming languages by offering a superior development experience and enhanced capabilities.

2. Product Overview & Goals

The **Next-Generation OS Development Language** is conceived as the definitive tool for building operating systems from bare metal. It addresses the inherent complexities and safety concerns of traditional OS development by offering a modern, opinionated approach to systems programming. The language's core philosophy centers on providing Python-like readability and developer ergonomics while delivering the raw performance and control traditionally associated with C and Assembly.

Key Goals:

- **Unification:** Provide a single language capable of handling all aspects of OS development, from bootloaders and kernel modules to user-space applications, eliminating the need for multi-language toolchains.
- **Performance:** Achieve bare-metal performance comparable to, or exceeding, C/C++ by leveraging the LLVM framework for aggressive optimizations.
- **Security & Safety:** Drastically reduce common OS vulnerabilities (e.g., buffer overflows, use-after-free) through compile-time memory safety guarantees and robust concurrency primitives.
- **Developer Experience:** Offer a “clean, high-level, easy as hell” syntax inspired by Python, significantly lowering the barrier to entry for OS development.
- **Hardware Control:** Provide direct, yet safe, access to hardware features, including memory-mapped I/O, CPU intrinsics, and interrupt handling, without resorting to inline assembly for common tasks.
- **Scalability:** Support the development of operating systems ranging from embedded systems to large-scale, multi-core, and distributed environments.

- **Ecosystem:** Foster a rich ecosystem with integrated tooling, a comprehensive standard library, and strong community support.

3. Key Features

3.1. Syntax & Semantics

The language prioritizes readability and conciseness, drawing inspiration from Python's design principles [1].

- **Indentation-based Block Structure:** Code blocks are defined by indentation, removing the need for curly braces (`{ }`) and reducing visual clutter.
- **Newline-based Line Termination:** Statements are terminated by newlines, eliminating semicolons (`;`).
- **Strong Typing with Inference:** The language is strongly typed, ensuring type safety at compile time. Robust type inference minimizes explicit type declarations, requiring them only when ambiguity exists (e.g., `let memory_buffer: [u8] = memory.allocate(1024)`).
- **Immutability by Default:** Variables are immutable by default, promoting safer programming practices. Mutability is explicitly opted into using the `mut` keyword (e.g., `let mut counter = 0`) [2].

3.2. Memory Management (Hybrid Model)

The language implements a sophisticated hybrid memory management model, ensuring safety by default while providing explicit control for low-level operations critical to OS development [1] [3].

- **Compiler-Enforced Safety:** For general operations, the compiler automatically manages memory lifecycles, employing strict borrowing rules similar to Rust to prevent common memory errors like segfaults and memory leaks [2].
- **Explicit Unsafe Control:** For direct hardware manipulation and performance-critical sections, the language provides explicitly scoped `@unsafe` blocks or tags. Within these blocks, developers can manipulate raw pointers and perform byte-level operations, with the understanding that they are responsible for memory safety [2]. This approach is analogous to Zig's manual allocator model, offering fine-grained control essential for kernel development [4] [5].

3.3. Data Structures: `struct` vs. `hwmap`

To clearly delineate application logic from hardware interaction, the language introduces two distinct data structure types [2]:

- **`struct` (Standard Structures):** Optimized by the compiler for general application logic. The compiler automatically handles padding and alignment for optimal performance and memory usage.
- **`hwmap` (Hardware Maps):** Designed for direct hardware interaction, `hwmap` guarantees perfectly packed, byte-aligned, and volatile memory access. This is crucial for device drivers interacting with components like network cards, USB controllers, and PCI devices, ensuring that memory layouts precisely match hardware registers and data formats.

3.4. Concurrency & Multi-core Safety

The language provides built-in, compile-time guarantees against deadlocks and race conditions, eliminating the need for external mutex libraries and simplifying concurrent programming [2].

- **shared Keyword:** Data accessed across multiple CPU cores must be explicitly declared `shared`.
- **Lock Blocks:** To modify `shared` data, developers must enter a `lock` block. The compiler automatically generates highly-optimized spinlocks (for kernel-level operations) or mutexes (for user-space) and guarantees their proper unlocking upon block exit, preventing common concurrency bugs.

3.5. OS & Hardware Operations

The language offers first-class support for low-level OS and hardware interactions, abstracting away the complexities of traditional methods [2].

- **Port I/O:** Built-in syntax for reading from and writing to CPU ports (e.g., `hw.inb(0x60)`).
- **Interrupts:** A declarative `@interrupt` attribute allows functions to be directly bound to hardware interrupts (e.g., `@interrupt(33) fn handle_keyboard_press():`).

3.6. Advanced CPU Control (Assembly Elimination)

This language aims to entirely eliminate the need for external `.asm` files by providing comprehensive built-in CPU control mechanisms [2].

- **Built-in CPU Intrinsics:** Direct language constructs for common CPU instructions (e.g., `cpu.halt()` for `hlt`, `cpu.disable_int()` for `cli`).
- **Inline Assembly:** For highly specific or architecture-dependent operations (e.g., context switches, manipulating `CR3` registers for paging), an `@unsafe` inline assembly feature is provided, allowing direct assembly insertion while clearly demarcating unsafe code.

4. Architecture

The language compiler will be built upon the robust and highly optimized **LLVM Framework** [1] [6]. This choice provides access to a mature, highly optimized backend capable of generating efficient machine code for a wide array of architectures.

1. **Front-End (Lexer & Parser):** Reads the custom syntax, performs lexical analysis, and constructs an Abstract Syntax Tree (AST).
2. **Semantic Analyzer:** Enforces type safety, applies memory borrowing rules, validates concurrency constructs, and ensures overall logical consistency of the code.
3. **IR Generator:** Translates the validated AST into LLVM Intermediate Representation (IR), enabling LLVM's powerful optimization passes.
4. **Backend (LLVM):** Utilizes LLVM's extensive backend to compile the IR into highly optimized, architecture-specific machine code (binary) for various target platforms, including x86, ARM, and potentially RISC-V.

5. Initial Scope (Phase 1 - Minimum Viable Compiler)

The first phase of development will focus on building a Minimum Viable Compiler (MVC) capable of demonstrating the core tenets of the language [1] [2]:

- **Lexing and Parsing:** Successfully parse a basic `start_os()` function, including simple variable declarations, control flow, and function calls.
- **AST Generation:** Generate a complete Abstract Syntax Tree for the parsed code.
- **LLVM IR Emission:** Emit valid and optimized LLVM IR for the `start_os()` function and its dependencies, demonstrating basic code generation and integration with the LLVM backend.
- **Basic Hardware Interaction:** Implement minimal support for `hwmap` and `hw.inb/outb` for a simple I/O operation.

6. Competitive Analysis

This language enters a landscape dominated by established systems programming languages. Its design aims to address their limitations and offer a superior alternative for OS development.

Feature / Language	C/C++	Rust	Zig	Nim	Next-Gen OS Language
Primary Focus	General Systems	Safe Systems	Low-level Systems	General Purpose	OS Development
Low-level Control	High	High (via <code>unsafe</code>)	High	Medium	Highest (Semantic APIs, Intrinsics)
Memory Safety	Manual	High (Borrow Checker)	Manual (Allocators)	Hybrid (GC + Manual)	Hybrid (Compiler-managed + Explicit Unsafe)
Concurrency Safety	Manual	High (Ownership)	Manual	Manual	Compile-time Guarantees
Performance	High	High	High	High	Bare-metal (LLVM Optimized)
Syntax Simplicity	Low	Medium	Medium	High	High (Python-inspired)
Hardware Access	Direct (Pointers)	Safe Abstractions	Direct (Pointers/Intrinsics)	FFI/Manual	Built-in Semantic APIs
Assembly Integration	Inline ASM	Inline ASM	Inline ASM	FFI/Inline ASM	Built-in Intrinsics & Inline ASM
Compiler Backend	GCC/Clang	LLVM	LLVM	C/C++/JS	LLVM

- **C/C++:** The traditional choice for OS development, offering unparalleled control and performance. However, it suffers from a lack of built-in memory safety, leading to a high incidence of critical bugs and security vulnerabilities. Its syntax can be verbose and error-prone for complex systems [7].
- **Rust:** Provides strong memory safety guarantees through its borrow checker and ownership system, making it excellent for preventing common bugs and security issues. However, its strictness can lead to a steeper learning curve and more complex code for certain low-level patterns, and its abstraction model can sometimes obscure direct hardware interaction [8].

- **Zig:** Emphasizes simplicity, explicit control, and direct hardware access. It provides manual memory management through allocators, offering fine-grained control without a garbage collector. While powerful, its philosophy prioritizes explicit control over automated safety features, requiring developers to be highly disciplined [4] [5].
- **Nim:** Aims for a Python-like syntax with C-like performance. It offers various memory management strategies, including garbage collection and manual control, providing flexibility. However, its primary focus isn't exclusively OS development, and its ecosystem for bare-metal programming is less mature than Rust or Zig [9] [10] [11].

Our **Next-Generation OS Development Language** aims to synthesize the best attributes of these languages while addressing their shortcomings. It offers the high-level readability and developer-friendliness of Python, the bare-metal performance and control of C/C++, and the memory and concurrency safety principles of modern languages like Rust, all underpinned by the robust LLVM infrastructure. This unique combination positions it as the ideal language for building secure, high-performance, and maintainable operating systems from the ground up.

7. Technical Specifications

7.1. Data Types

- **Primitive Types:** `int` (64-bit signed integer by default), `u8`, `u16`, `u32`, `u64` (unsigned integers), `i8`, `i16`, `i32`, `i64` (signed integers), `float` (64-bit floating-point by default), `f32`, `f64` (single and double-precision floats), `bool` (`true`, `false`), `str` (UTF-8 string literals), `char` (single character) [2].
- **Pointers:** Explicit pointer types for raw memory access, strictly confined to `@unsafe` blocks.
- **Arrays & Slices:** Statically sized arrays (e.g., `[u8; 6]`) and dynamically sized slices for contiguous memory regions.
- **Structs & Enums:** User-defined composite types for data structuring and algebraic data types.

7.2. Control Flow

- **Conditional Statements:** `if` / `elif` / `else` constructs with indentation-based blocks [2].
- **Looping Constructs:** `for` loops for iteration over collections, `while` loops for condition-based iteration [2].
- **Pattern Matching:** Advanced `match` statements for expressive control flow and destructuring, enabling robust error handling and state management.

7.3. Functions

- **Function Declaration:** `fn function_name(param1: Type1, param2: Type2) → ReturnType`: [2]. Return types are required for functions returning a value; otherwise, they are inferred as `void`.

- **Methods:** Functions associated with `struct` or `hwmap` types.
- **Closures:** Anonymous functions with captured environments, supporting higher-order programming paradigms.

7.4. Modules & Imports

- **Module System:** A hierarchical module system for organizing code into logical units, promoting modularity and reusability.
- **Import Mechanism:** `import module_name` for bringing external code and functionalities into scope [2].

8. Standard Library Ecosystem

Beyond the bare-metal core, a comprehensive and minimal standard library will be developed to facilitate high-level OS development [2]:

- **`std.graphics` (Frontend):** Provides tools for graphical output, including drawing windows, text rendering, and 2D UI elements directly to the screen buffer.
- **`std.fs` (Database/Storage):** A built-in filesystem module enabling creation of files and folders, and read/write operations to persistent storage, effectively acting as the OS registry and database.
- **`std.net` (Backend):** Implements TCP/UDP networking stacks, allowing the OS to support web browsing, act as a server, and engage in network communication.

9. Compiler Pipeline & Tooling

The compiler toolchain is designed for a seamless developer experience, from code to a bootable OS [2]:

1. **Lexical Analysis & Parsing:** Transforms source code into an Abstract Syntax Tree (AST).
2. **Semantic Analysis:** Performs type checking, memory safety validation, concurrency checks, and other semantic validations.
3. **IR Generation:** Translates the AST into LLVM Intermediate Representation.
4. **Machine Code Emission:** LLVM optimizes the IR and generates architecture-specific object files (`.o`).
5. **Linking:** Links the generated object files with a bootloader script to produce a bootable image (`.bin` or `.iso`).

9.1. Editor Support & Execution

- **Language Server Protocol (LSP):** The compiler toolchain will feature a native LSP implementation, providing rich editor support (syntax highlighting, auto-complete, error checking) for VS Code and other compatible IDEs.

- **Integrated Virtual Machine Execution:** The `compiler run os_kernel.os` command will automatically compile the OS code and launch it within a local **QEMU Virtual Machine**, allowing developers to instantly test and debug their OS directly within their development environment.

10. Future Considerations & Roadmap

- **Advanced Concurrency:** Explore advanced concurrency models and distributed systems support.
- **Formal Verification:** Investigate integration with formal verification tools to prove correctness of critical OS components.
- **Hardware Abstraction Layer (HAL):** Develop a robust HAL to simplify porting the OS to new hardware platforms.
- **Package Manager:** Create a dedicated package manager for easy distribution and management of libraries and modules.
- **Debugging Tools:** Enhance debugging capabilities with advanced features like kernel-level debugging and hardware-assisted debugging.

11. References

[1] OS Language Design Specification (User Provided Document) [2] OS Programming Language - Official Specification (User Provided Document) [3] Possible to mix garbage collection and manual memory management? - Stack Exchange (<https://langdev.stackexchange.com/questions/2956/possible-to-mix-garbage-collection-and-manual-memory-management>) [4] Why Zig When There is Already C++, D, and Rust? - Ziglang (https://ziglang.org/learn/why_zig_rust_d_cpp/) [5] Zig's design is so radical, that it completely rethinks how low-level ... - Hacker News (<https://news.ycombinator.com/item?id=26374647>) [6] A Practical Guide for Building a Compiler With LLVM - Reddit (https://www.reddit.com/r/Compilers/comments/1edl044/a_practical_guide_for_building_a_compiler_with/) [7] System Design Introduction - LLD & HLD - GeeksforGeeks (<https://www.geeksforgeeks.org/system-design/getting-started-with-system-design/>) [8] Zig, Rust, Go?! I tried 3 low-level languages and here's what I'm ... - dev.to (https://dev.to/dev_tips/zig-rust-go-i-tried-3-low-level-languages-and-heres-what-im-sticking-with-4gpp) [9] Nim's Memory Management (<https://nim-lang.org/docs/mm.html>) [10] Memory management in Nim programming language - Medium (<https://medium.com/@m.matyunin/memory-management-in-nim-programming-language-1454a798cd18>) [11] Choosing Nim out of a crowded market for systems programming ... - Nim Forum (<https://forum.nim-lang.org/t/9655>)